面向对象程序设计

考核方式
- 期末考试 (70%)
- 课外工程实践项目 (15%)
- 考勤（5%）
- 作业/实验报告(10%)

上课
时间：周二9-10
地点：FJ203

上机
时间：周四11-13（2/3/5/7/8/9）
地点：D413（124机位）

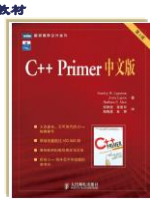
课程资料
内容：作业、课程项目
课件、上机习题
位置：网上教学系统

朱晓强
xqzhu@shu.edu.cn
通信楼705
答疑：周二7-8节

课程引言－教材

- 教 材

- 参考教材



课程引言－学习方法

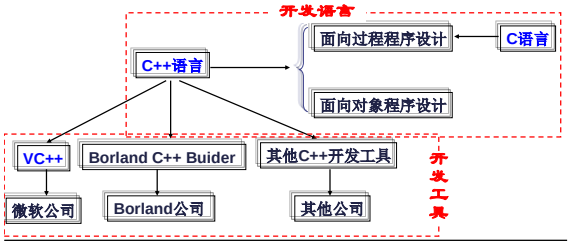
- 学习程序设计最好的方法之一：阅读源代码
- 学习编程的秘诀是：抄—运行—改—运行—扩充—运行—独立实现—运行
- 切忌浮躁
- 能看懂的内容，请仔细看；看不懂的内容，请硬着头皮看。
- 多遍阅读教材。

第一章 绪论

1. 面向过程程序设计的基本思想
2. 从面向过程到面向对象
3. 面向对象程序设计的几个基本概念
4. 面向对象程序设计的基本特征

1. 面向过程程序设计基本思想

C、C++及VC++之间的关系

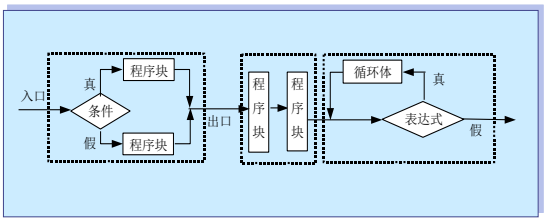


✓C++是C的一个超集，它几乎保留了C的所有的特性，并对C作了扩展，增加了面向对象功能。

✓VC++是Microsoft公司推出的一款基于C++语言的软件开发工具。

Visual Studio 6： vc6
Visual Studio 2002： vc7
Visual Studio 2003： vc7.1
Visual Studio 2005： vc8
Visual Studio 2008： vc9
Visual Studio 2010： vc10
Visual Studio 2012： vc11
Visual Studio 2013： vc12
Visual Studio 2015： vc14
Visual Studio 2017： vc15

1. 面向过程程序设计基本思想



只允许顺序、分支、循环三种结构组成结构化程序。
每一种结构只能一个入口，一个出口。
自顶向下，逐步求精。
易编程、易维护、提高了软件开发效率和质量。

1. 面向过程程序设计基本思想

例：自我介绍，whoami

定义一个结构体PERSON，包含成员name；生成一个结构体变量，调用函数设置他的姓名；调用函数whoami打印他的姓名.

```
struct Person{
      char name[20];
      …;
   };

void whoami(struct Person);

void main(){
   struct Person mike;
   strcpy(mike.name, “mike”);
   whoami(mike);
}

void whoami(struct Person p)
{
   printf(“%s”, p.name);
}
```


1. 面向过程程序设计基本思想

面向过程程序设计：

✓**程序＝算法＋数据结构**

对操作的描述。即操作步骤

指定数据的类型和数据的组织形式

```
struct Person{
      char name[20];
      …;
   };

void whoami(struct Person);

void main(){
   struct Person mike;
   strcpy(mike.name, “mike”);
   whoami(mike);
}

void whoami(struct Person p)
{
   printf(“%s”, p.name);
}
```


1. 面向过程程序设计基本思想

✓过程式程序设计是一种以功能为中心、基于功能分解的程序设计方法。

✓一个过程式程序由一些子程序构成，每个子程序对应一个子功能，它实现了功能抽象。子程序描述了一系列的操作，它是操作的封装体。

✓过程式程序的执行过程体现为一系列的子程序调用。在过程式程序中，数据处于附属地位，它独立于子程序，在子程序调用时作为参数传给子程序使用。

```
struct Person{
      char name[20];
      …;
   };

void whoami(struct Person);

void main(){
   struct Person mike;
   strcpy(mike.name, “mike”);
   whoami(mike);
}

void whoami(struct Person p)
{
   printf(“%s”, p.name);
}
```


1. 面向过程程序设计基本思想

✓程序员只能按过程式程序设计的逻辑结构（而不是按问题本身的逻辑结构）去描述问题；因此，问题的求解实际上是一种过程的抽象。

✓也就是说，程序员必须在实际问题模型（问题空间）和机器模型（解空间）之间进行转换。

✓然而，当实际问题发生改变时，程序中各种成分也随之改变。其中：

- 功能：很容易变
- 过程执行顺序：很容易变
- 接口：很容易变
- 数据：很容易变

代码的重用性差！

```
struct Person{
      char name[20];
      …;
   };

void whoami(struct Person);

void main(){
   struct Person mike;
   strcpy(mike.name, “mike”);
   whoami(mike);
}

void whoami(struct Person p)
{
   printf(“%s”, p.name);
}
```


第一章 绪论

1. 面向过程程序设计的基本思想
2. 从面向过程到面向对象
3. 面向对象程序设计的几个基本概念
4. 面向对象程序设计的基本特征

2. 从面向过程到面向对象

□从客观世界到面向对象程序设计

✓例：人

静态特征(属性attribute)

姓名、性别、年龄、身高等

动态特征(行为behavior)

吃饭、睡觉、说话、走路等

✓例：汽车

静态特征(属性attribute)

品牌、生产厂、型号、颜色等

动态特征(行为behavior)

行驶、转弯、鸣笛、刹车等

抽象

✓**抽象**是先注意问题的本质及描述，其次是实现过程或细节。

✓**两个抽象**：

1、数据抽象——描述某类对象的属性或状态。

2、代码抽象——描述某类对象的共有的行为特征或功能。

2. 从面向过程到面向对象

例：自我介绍，whoami

定义一个类IntroduceMyself，包含属性name和行为whoami；生成一个名为Me的对象(mike)；给Me发送一个消息，让他自己通过行为whoami说出他的名字.

```
class IntroduceMyself
{
  public:
    char name[20];  //静态特征
    IntroduceMyself (char *na )
      {strcpy(name, "mike");}
    void   whoami(){cout<<name;};
};

void main()
{
 IntroduceMyself Me("mike");
 Me.whoami(); //给mike发送一个消息，让他自己通过行为whoami说出他的名字
}
```


2. 从面向过程到面向对象

面向过程：

```
struct Person{
      char name[20];
      …;
   };

void whoami(struct Person);

void main(){
   struct Person mike;
   strcpy(mike.name, "mike");
   whoami(mike);
}

void whoami(struct Person p)
{
   printf("%s", p.name);
}
```

属性和行为分离

面向对象：

```
class IntroduceMyself{
public:
   char name[20];  //静态特征
   IntroduceMyself (char *name)
     {strcpy(name, "mike");}
  void   whoami(){cout<<name;};
   …;   //动态特征
};
```

属性和行为关联

```
void main(){
 IntroduceMyself mike("mike");
 mike.whoami(); //给mike发送一个消息，让他自己通过行为whoami说出他的名字
}
```


2. 从面向过程到面向对象

✓面向对象程序设计以对象为中心。

✓一个面向对象程序由一些对象构成，对象是由一些数据及可施于这些数据上的操作所构成的封装体。

✓对象的特征由相应的类来描述。

✓面向对象程序的执行过程体现为各个对象之间相互发送和处理消息.

面向对象：

```
class IntroduceMyself{
public:
   char name[20];  //静态特征
   void   whoami(){cout<<name;};
   …;   //动态特征
};
```

属性和行为关联

```
void main(){
 IntroduceMyself mike("mike");
 mike.whoami(); //给mike发送一个消息，让他自己通过行为whoami说出他的名字
}
```


2. 从面向过程到面向对象

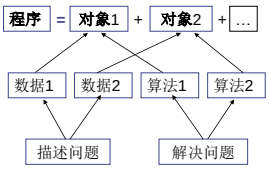
面向过程：

程序＝算法＋数据结构

对操作的描述。即操作步骤

指定数据的类型和数据的组织形式

面向对象：



第一章 绪论

1. 面向过程程序设计的基本思想
2. 从面向过程到面向对象
3. 面向对象程序设计的几个基本概念
4. 面向对象程序设计的基本特征

3. 面向对象程序设计的几个基本概念

对象（Object）

类（Class）

消息（Message）

方法（Method）

3. 面向对象程序设计的几个基本概念—对象

- 面向对象技术就是以要解决的问题领域中所涉及到的各种**对象**为主要考虑因素。
- 计算机世界中的对象是现实世界对象的模型化，所以它也应该具有状态和行为。
- 在计算机程序环境中：

对象 { 状态 — 【属性变量，有数值】
行为 — 【方法函数，执行语句】

- 因此，解空间中的对象是由描述该对象属性的数据以及可以施加于这些数据上的操作（方法代码）所组成。

3. 面向对象程序设计的几个基本概念—对象

- 例：“电梯”对象

电梯
- 电梯编号
- 最大载重量
- 空间大小
- 电梯门的当前状态（开或关）
- 电梯当前所在楼层
- 开门
- 关门
- 上升
- 下降
- 停靠
- 检测载重量

在计算机中，同一类型的一个对象与另一个对象区分：对象的属性取值不同。不同类的对象属性和行为均不相同。

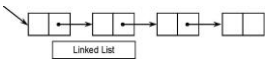
3. 面向对象程序设计的几个基本概念—对象

一个对象用来描述一个实体(entity)，这个实体可以是物理实体、概念上的实体或者是软件。

物理实体：卡车

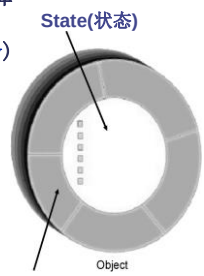
概念实体：化学反应过程

软件实体：



3. 面向对象程序设计的几个基本概念—对象

- ✓ 对象是现实世界中某个实体在计算机逻辑中的映射和描述。
- ✓ 对象在一个明确的边界(boundary)里封装了状态(*state*)和行为(*behavior*)。
  - 状态通过属性来表示
  - 行为通过方法表示



Behaviors(行为)

3. 面向对象程序设计的几个基本概念—对象

对象的状态(state)

状态(state)描述的是对象在不同时刻所表现出来的一种状况(condition)或情形(situation)：例如，满足某种条件，在做某件事情或者等待某个事件的发生。

通常对象的状态随时间的变化而改变。

Name: J Clark
Employee ID: 567138
Date Hired: July 25, 1991
Status: Tenured （终身职位）

Discipline(学科): Finance(金融学)
Maximum Course Load(最大课程负担): 3 classes

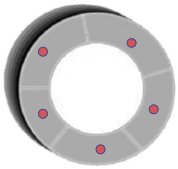
Professor Clark

3. 面向对象程序设计的几个基本概念—对象

对象的行为(behavior)

对象的行为确定了一个对象如何行动(acts)，如何对事件进行响应(reacts)。

Clark老师的行为
- 提交期末成绩
- 接受课程分配
- 休假年
- 。。。



3. 面向对象程序设计的几个基本概念—类

- 类是对具有相同属性和操作的对象集合的描述
  - 对象是类的实例
- 类是一种抽象，因为它
  - 强调相关的本质特征
  - 舍去其他的无关特征

对象 { 状态 / 行为 }

类 { 属性【变量】 / 方法【函数执行代码】 }

3. 面向对象程序设计的几个基本概念—类

类的属性(Attributes)

属性 {

Student
- name - address - studentID - dateOfBirth

}

3. 面向对象程序设计的几个基本概念—类

类的方法(Method)

Operations {

Student
+ get tuition() + add schedule() + get schedule() + delete schedule() + has prerequisites()

}

3. 面向对象程序设计的几个基本概念—类

例：课程类：course

Class
Course

Properties
Name
Location
Days offered
Credit hours
Start time
End time

Methods
Add a student
Delete a student
Get course roster (花名册)
Determine if it is full

3. 面向对象程序设计的几个基本概念—类

类 {

属性 {

Professor
- name - employeeID : UniqueId - hireDate - status - discipline - maxLoad
+ submitFinalGrade() + acceptCourseOffering() + setMaxLoad() + takeSabbatical()

}

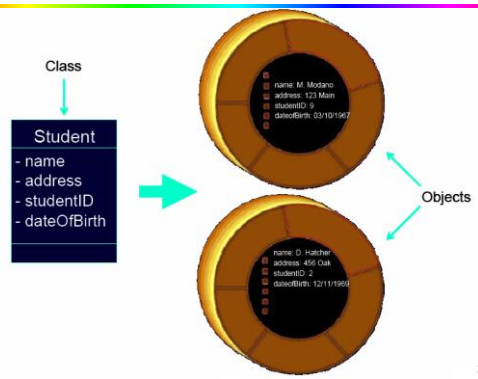
方法 }

3. 面向对象程序设计的几个基本概念—类

对象 和类的关系

- 类是对象的抽象定义，是创建对象的模板
- 类不是对象集合，是对象的数据类型

3. 面向对象程序设计的几个基本概念—类



3. 面向对象程序设计的几个基本概念—消息

•面向对象程序设计技术必须提供一种机制，允许一个对象与另一个对象的交互。这种机制叫消息传递。

•一个对象向另一个对象发出的请求称为“消息”

消息=接收对象名+调用操作名+必要的参数

```
class IntroduceMyself{
public:
   char name[20];
   void   whoami() //方法（界面）
      {cout<<name;} //方法(方法体)
   …;
};

void main(){
 IntroduceMyself mike("mike");
 mike.whoami(); //消息
}
```


3. 面向对象程序设计的几个基本概念—方法

- 对象所能实现的行为（操作），在程序设计方法中称为方法，通过调用相应的函数来实现。
- 方法包括：界面　和　方法体
- 消息和方法的关系：对象根据接受到的消息，调用相应的方法；反过来，有了方法，对象才能响应相应的消息。

```
class IntroduceMyself{
public:
   char name[20];
   void   whoami() //方法（界面）
      {cout<<name;} //方法(方法体)
   …;
};

void main(){
 IntroduceMyself mike("mike");
 mike.whoami(); //消息
}
```


第一章 绪论

1. 面向过程程序设计的基本思想
2. 从面向过程到面向对象
3. 面向对象程序设计的几个基本概念
4. 面向对象程序设计的基本特征

4. 面向对象程序设计的基本特征

抽象（Abstraction）

封装（Encapsulation）

继承（Inheritance）

多态（Polymorphism）

3. 面向对象程序设计的基本特征—抽象

抽象

✓将有关事物的共性归纳、集中的过程。

✓通过特定的实例（对象）抽取共同性质后形成概念的过程。

数据抽象
行为抽象

“学生”对象：

相同特性或主要特性：
- 姓名
- 性别
- 年龄
- 专业
- …

忽略不同点或次要特性：
- 长相差异
- 性格差异
- …

学生成绩管理中的对象共同的行为：
- 数据录入
- 修改
- 查看
- …

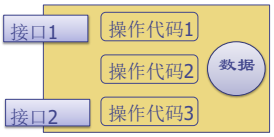
3. 面向对象程序设计的基本特征—封装

封装

✓在面向对象程序设计中，封装是指把数据和实现操作的代码集中起来放在对象内部，并尽可能隐蔽对象的内部细节。

◈ 封装实例

洗衣机对象



3. 面向对象程序设计的基本特征—封装

封装

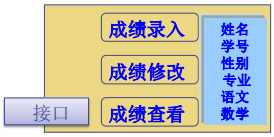
封装具有两方面含义：

◈将有关数据和操作代码封装在一个对象中，各个对象相对独立、互不干扰；

◈对象中某些数据和操作代码对外隐蔽，即隐蔽其内部细节，只留下少量接口，以便与外界联系，接受外界消息。

◈ 封装实例

学生成绩管理



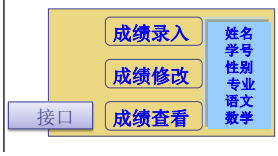
3. 面向对象程序设计的基本特征—封装

封装的本质：隐蔽实现

- 通过封装，使得程序员分为：
  - 一类创建者：专注于实现类
  - 一客户程序员：使用类的用户，专注于使用类
- 优点：
  - 一 避免客户程序员插手他们不应当接触的部分。
  - 一 实现者可以随时修改被隐藏的内部工作方式，而不影响客户程序员(的使用)。

◈ 封装实例

学生成绩管理



3. 面向对象程序设计的基本特征—继承

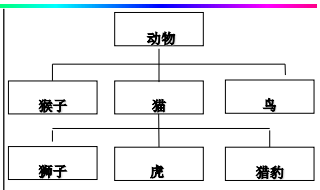
继承

✓继承所表达的是类之间相关的关系。

✓这种关系使得某类对象可以继承另外一类对象的特征和能力。

✓继承的作用是什么？

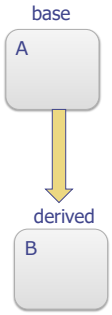
1. 避免公用代码的重复开发，减少代码和数据冗余；
2. 通过增强一致性来减少模块间的接口和界面。



3. 面向对象程序设计的基本特征—继承

继承

假设有两个类A和B，若类B继承类A，则属于类B中的对象具有类A的一切特征（包括属性和操作），这时，我们称被继承类A为基类或父类或超类；而称继承类B为类A的派生类或子类。同时，我们还可以说，类B是从类A中派生出来的。



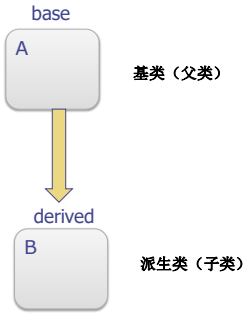
基类（父类）

派生类（子类）

3. 面向对象程序设计的基本特征—继承

继承

- 如果类B是类A的派生类，那么，在构造类B的时候．我们不必重新描述A的所有特征，只需让它继承类A的特征，然后描述与基类A不同的那些特性。也就是说，类B的特征由继承来的和新添加的两部分特征构成。

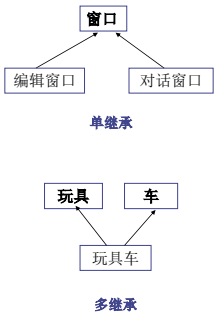


3. 面向对象程序设计的基本特征—继承

继承分类

➢1、单继承是指每个派生类只直接继承了一个基类的特征。动物链就是一个单继承的实例。

➢2、多继承是指多个基类派生出一个派生类的继承关系，多继承的派生类直接继承了不止一个基类的特征。



3. 面向对象程序设计的基本特征—多态性

多态性

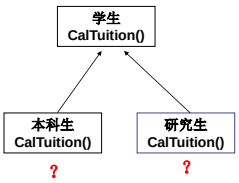
✓指不同的对象在接收到相同的消息时产生的多种不同的行为方式

✓C++语言支持两种多态性，即编译时的多态性和运行时的多态性。

✓编译时的多态性是通过重载来实现的，

✓运行时的多态性是通过虚函数来实现的。

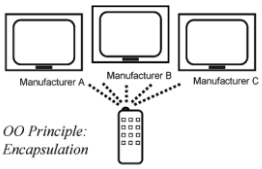
void EatBeef();——void Eat (Beef);
void EatFish();—— void Eat (Fish);
void EatChicken(); — void Eat (Chicken);



3. 面向对象程序设计的基本特征—多态性

多态性

- 多态性是能够把多种不同的实现隐藏在一个单一的接口后面的能力。



总结：

✓面向对象程序设计的基本思想

✓几个重要概念：类、对象、消息、方法

✓面向对象程序设计的几个基本特征：抽象、封装、继承、多态

END